

Construction Replay Transcript

EDI 835 Payment Integrity — Design · 2026-08-13 · what you'll hear in documents/construction-replay.mp4

What this interface does

Ingest ERA 835s, enrich each claim payment with open-AR lineage from SQL Server, bucket matched vs exception, and emit an underpay alert CSV for shortfalls finance can act on. Demo only — not a full posting engine.

Who it's for

- Sources: Remit drop folder (input/inbound/*.edi)
- Destinations: output/matched/, output/exceptions/, output/underpay/underpay_alerts.csv, OpenAR status updates
- Demo vs production: Demo only — synthetic 835 + AR, no PM/EHR posting, no SNIP Types 1–7 for 835
- Anyone watching the construction video or reading this transcript

Systems involved

- Sources: Remit drop folder (input/inbound/*.edi)
- Destinations: output/matched/, output/exceptions/, output/underpay/underpay_alerts.csv, OpenAR status updates
- Demo vs production: Demo only — synthetic 835 + AR, no PM/EHR posting, no SNIP Types 1–7 for 835

Documents we also produce

EDI835_Payment_Integrity_Capability_Brief.pdf · ready

Short overview of what the interface does — shareable without reading route XML.

EDI835_Payment_Integrity_Test_Plan.pdf · ready

Scenarios that prove the interface works end-to-end.

Module documentation pack (12 deep-dive PDFs) · ready

PilotFish product PDFs for the modules used in this build.

EDI835_Payment_Integrity_V2_Route_Diagrams.pdf · ready

V2 route diagrams for visual review of module topology.

construction-replay.mp4 + this transcript · ready

Narrated walkthrough of building the routes, then a live Demo-tab test when inject is available.

Narration

Spoken lines from the construction video (setup, build-replay, live test, closing).

Setup

Welcome

Welcome to the PilotFish demo: EDI 835 Payment Integrity. Ingest ERA 835s, enrich each claim payment with open-AR lineage from SQL Server, bucket matched vs exception, and emit an underpay alert CSV for shortfalls finance can act on. PilotFish supports the whole flow out of the box.

Pipeline overview

Before we wire anything, here's the flow end to end. Ingest ERA 835s, enrich each claim payment with open-AR lineage from SQL Server, bucket matched vs exception, and emit an underpay alert CSV for shortfalls finance can act on.

External systems

And here's what sits around the routes. SQL Server, PilotFish eiPlatform, and Demo Web UI. All of those are spun up in docker images for this demo.

1 - Poll And Score Remits

First route

Ingest ERA 835s, enrich each claim payment with open-AR lineage from SQL Server, bucket matched vs exception, and emit an underpay alert CSV for shortfalls finance can act on. We'll build that in 2 routes. Starting with a blank canvas on the first route.

Poll EDI 835 Files

Here's a directory listener — it watches a local folder. That folder is `/opt/pilotfish/input/inbound`. We're only taking `.edi,835,txt` files. It checks about every 5 seconds. After pickup we move the file aside so the folder stays clean.

Fork - XPath

This XPath fork splits the file so each transaction set becomes its own message.

Extract CLP Remit Fields

XPath evaluation pulls the claim control number, paid amount, and charge off the EDI XML.

Debug Write Forked Transaction

This writes a debug copy of the XML so we can inspect what the route saw. That lands under `/opt/pilotfish/output/debug`.

Lookup Open AR

This is a SQL lookup — we query `Edi835PaymentIntegrity` over JDBC using a key from the transaction.

Extract Open AR Fields

Another XPath evaluation — this time from the SQL result, so we have expected paid and the patient name.

Compose Patient Name

Then we stitch last name and first name into one patient name on the transaction.

Build Remit Decision XML

Now for the mapping — we're using the stock XSLT processor with `build-remit-decision.xslt`. It maps `ClaimControlNumber`, `PatientName`, `ChargeAmount`.

Debug Write Remit Decision

This writes a debug copy of the XML so we can inspect what the route saw. That lands under `/opt/pilotfish/output/debug`.

Pretty-Print XML

This processor pretty-prints the XML — line breaks and indent — so the file on disk is readable.

Stage Remit Decision

Then we write the file out to `/opt/pilotfish/output/staged-decisions`.

2 - Route Remit Decisions

Next route

Next route — blank canvas again, then the modules one at a time.

Staged-folder listener

Here's a directory listener — it watches a local folder. That folder is `/opt/pilotfish/output/staged-decisions`. We're only taking `.xml` files. It checks about every 3 seconds.

Extract Decision Attributes

XPath evaluation copies the match bucket and underpay flag off the decision XML so the router can fan out.

Conditional Router

Then a conditional router — it uses XPath to pick which path this transaction takes.

Pretty-Print Matched XML

This processor pretty-prints the XML — line breaks and indent — so the file on disk is readable.

Mark Open AR Matched

Then we update that row in `Edi835PaymentIntegrity` so the status matches the decision.

Pretty-Print Exception XML

This processor pretty-prints the XML — line breaks and indent — so the file on disk is readable.

Mark Open AR Exception

Then we update that row in `Edi835PaymentIntegrity` so the status matches the decision.

Map Decision To Underpay CSV

Now for the mapping — we're using the stock XSLT processor with `decision-to-underpay-csv.xslt`. It maps `ClaimControlNumber`, `PatientName`, `ExpectedPaid`.

Write Matched

Then we write the file out to `/opt/pilotfish/output/matched`.

Update AR Matched

The SQL update already ran on this path, so we finish on a null transport — nothing else to send.

Write Exception

Then we write the file out to `/opt/pilotfish/output/exceptions`.

Update AR Exception

The SQL update already ran on this path, so we finish on a null transport — nothing else to send.

Append Underpay Alert CSV

Then we write the file out to /opt/pilotfish/output/underpay.

Live test

After construction, the video switches to the Demo tab and runs a live inject when the UI supports it.

Prove it works

Routes are built. Let's switch to the Demo tab and prove it works.

Submit a sample

I'll submit this sample from the Demo tab — same as an operator would.

Waiting for the routes

Give the routes a moment to pick up the file and write the output.

OpenAR

There it is — output on the results panel.

Submit another sample

I'll send it one more time so we can watch it land again.

Waiting for the routes

Same path — the second result should show up here.

OpenAR

And there's the second result.

Closing

Demo complete

That's the demo — ingest ERA 835s, enrich each claim payment with open-AR lineage from SQL Server, bucket matched vs exception, and emit an underpay alert CSV for shortfalls. Thanks for choosing PilotFish.